Phase 0

Pedro Pereira edited this page on Mar 6, 2025 · 11 revisions

Perform the following operations **only once** for each group:

1. Clone the group's repository - `git clone <group-repo-url>`.
2. Add a remote pointing to the `common` repo - `git remote add common https://github.com/isel-leic-ls/2425-2-common.git`.
3. Pull the contents from the `common` repo - `git pull common main`.
4. If and only if error of "unrelated histories" do:

   `git config pull.rebase false`

   `git pull common main --allow-unrelated-histories`
5. Remove the current content of `README.md` and add the number and name of each group member (use the markdown format).
6. Create a commit containing those changes.
7. Push the commits to the group's repository - `git push origin main`.

Correct errors and add tests (multiple group members should coordinate these tasks)

1. Ensure you have an updated repo.
2. Execute the unit tests using Gradle and check that there is an error.
3. Create a project in IntelliJ using the recommendations of the previous classes.
4. Execute the same testes now using the IntelliJ IDE.
5. Correct the cause of the error and check that all tests are now correctly executed.
6. Add more tests if necessary.
7. Check, using the Gradle build script at the command line, that all the tests are executed with success.
8. Publish all the changes (if any) to the local and group repository. Make one or more commits for this task.

Database configuration and connectivity test

1. Create a database in the chosen DB Management system (i.e. Postgres) on your machine. Add a table to represent *students*.
2. Make a set of tests to check that your system can read (i.e. `SELECT`), update (i.e. `INSERT` and `UPDATE`) and delete (i.e. `DELETE`) over the previous table, using the JDBC interface. Assign different tests to each member of the group.
3. Publish the changes to the group repo.

Create tag

1. Create a *tag* with the identifier `0.0.0` and publish it to the group repo

- `git tag -a 0.0.0 -m "Version 0.0.0"`.
- `git push origin 0.0.0`.
- Performed only once per

Pages 20

Find a page or section...

Home
Calendar
Demonstration
E Learning Guidelines
Introduction
JDBC
LEIC41D - Groups
LEIC41D - LEIC42D - LEIC43D...
LEIC41N - Groups
LEIC42D - Groups
LEIC43D - Groups
LEIC44D - Groups
Phase 0
Phase 1
Phase 2
Show 5 more pages...

+ Add a custom sidebar

Clone this wiki locally

https://github.com/isel-leic-ls/2425

+ Add a custom footer

© 2026 GitHub, Inc.

Phase 1

fbfreitas edited this page on Mar 17, 2025 · 21 revisions

Phase 1

Goals

The Software Laboratory project consists of the analysis, design and implementation of an information system to manage padel courts. Its development is divided into 4 phases, with incremental requirements published before the beginning of each phase.

Domain

- A *user* is characterized by a unique number (uid), a name and a unique email.
- A *club* is characterized by a unique number (cid), a unique name and is associated to a user (owner).
- A *court* is characterized by a unique number (crid), a name and is associated to a *club*.
- A *rental* is characterized by a unique number (rid), a date, a duration in hours and is associate to a user and a court

NOTE: All courts are avaliable 24 hours

Phase 1 requirements

Development of the backend service. The service must expose an HTTP API to frontend client applications. The API should provide the following functionalities:

User Management

- Create a new user, given the following parameters:
  - `name` - the user's name
  - `email` - the user's unique email.

  Returns the user's token and the user's identifier (uid).
- Get the details of a user.

Club Management

- Create a club, given the following parameters:
  - `name` - the club name

  Returns the club identifier (cid).
- Get the details of the club.
- Get a list with the clubs

Court Management

- Creates a new court, given the following parameters:
  - `name` - court name
  - `cid` - club identifier

  Returns the court identifier (crid).
- Get the detailed information of a court.
- Get a list with the courts given the club identifier.

Rental Management

- Create a rental, given the following parameters:
  - `cid` - club identifier
  - `crid` - court identifier
  - `date` - rental date with initial hour
  - `duration` - duration in hours

  Returns the rental identifier (rid).
- Get the detailed information of a rental.
- Get a list with the rentals, given the following parameters:
  - `cid` - club identifier
  - `crid` - court identifier
  - `date` - date (Optional)
- Get a list with the rentals of a user
- Get a list with the avaliable hours to rent a court of a club, given the following parameters:
  - `cid` - club identifier
  - `crid` - court identifier
  - `date` - date

Paging

All Web API `GET` operations that return a sequence should support paging, i.e., the ability to return a subsequence from the overall sequence. This paging is defined by two parameters:

- `limit` - length of the subsequence to return.
- `skip` - start position of the subsequence to return.

For example, the request `GET /rentals?skip=6&limit=3` returns the rentals subsequence starting at the seventh rental and ending on the ninth rental.

NOTE: For all operations that require authentication, a user token must be sent in the Authorization header using a Bearer Token. This token is generated at user creation, and consists of a UUID string (you can use Uuid class from `kotlin.uuid` package for ID generation). Students should determine which functionalities require authentication and authorization. For example, only authenticated users can create resources, and only the club owner can add courts.

Non-functional requirements

Pages 20

Find a page or section…

Home
Calendar
Demonstration
E Learning Guidelines
Introduction
JDBC
LEIC41D - Groups
LEIC41D - LEIC42D - LEIC43D…
LEIC41N - Groups
LEIC42D - Groups
LEIC43D - Groups
LEIC44D - Groups
Phase 0
Phase 1
Phase 2

Show 5 more pages…

+ Add a custom sidebar

Clone this wiki locally

https://github.com/isel-leic-ls/2425

The backend must be developed with Kotlin technology. To handle/receive HTTP requests, the HTTP4K library must be used. The body serialization/deserialization should be done using the kotlinx.serialization library- You can see a functional usage example of both libraries here. The data that are specific to the application, which can be created, altered and deleted must be **stored in a Postgres database**. Tests run using data stored in memory, not the database. The following non-functional requirements will be highly valued.

- Readability.
- Testability, including on machines not belonging to the development team.

NOTE: Tests are mandatory

Guidelines about design and implementation

At the **beginning** of the development, the server application should store the data in memory and the solution should start with 4 files:

- `courtsServer.kt` - file that constitutes the entry point to the server application
- `courtsWebApi.kt` - implementation of the HTTP routes that make up the REST API of the web application
- `courtsServices.kt` - implementation of the logic of each of the application's functionalities
- `courtsDataMem.kt` - access to data, stored in memory.

The dependencies between these are as follows:

```
courtsServer.kt -> courtsWebApi.kt -> courtsServices.kt -> courtsDataMem.kt
```

During the development of this phase other files can (and should) be created, in order to structure the code in a more logical and readable form. Solutions with large code files are highly discouraged.

Report and HTTP API Documentation

- The phase 1 delivery must include a technical report, in the docs repository folder, using the following template.
- It also has to include the HTTP API documentation required for a frontend client application to use this API. This documentation should not include any information about the internal backend implementation. Students are encoraged to follow OpenAPI specification.

Delivery date

The project must be delivered until 29 March (end of week 6), via the creation of a 0.1.0 tag on the GitHub repository.

+ Add a custom footer

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

Phase 2

Pedro Pereira edited this page on Apr 2, 2025 · 10 revisions

Introduction

This document describes the requirements for the second phase of the Software Laboratory project.

Requirements

Additional Operations

- Delete a rental
- Update a rental
  - `date` - rental date
  - `duration` - rental duration

Single Page Application

The main requirement for the second phase is the delivery of a Single Page Application (SPA) to provide a Web User Interface to the `GET` operations developed in the first phase.

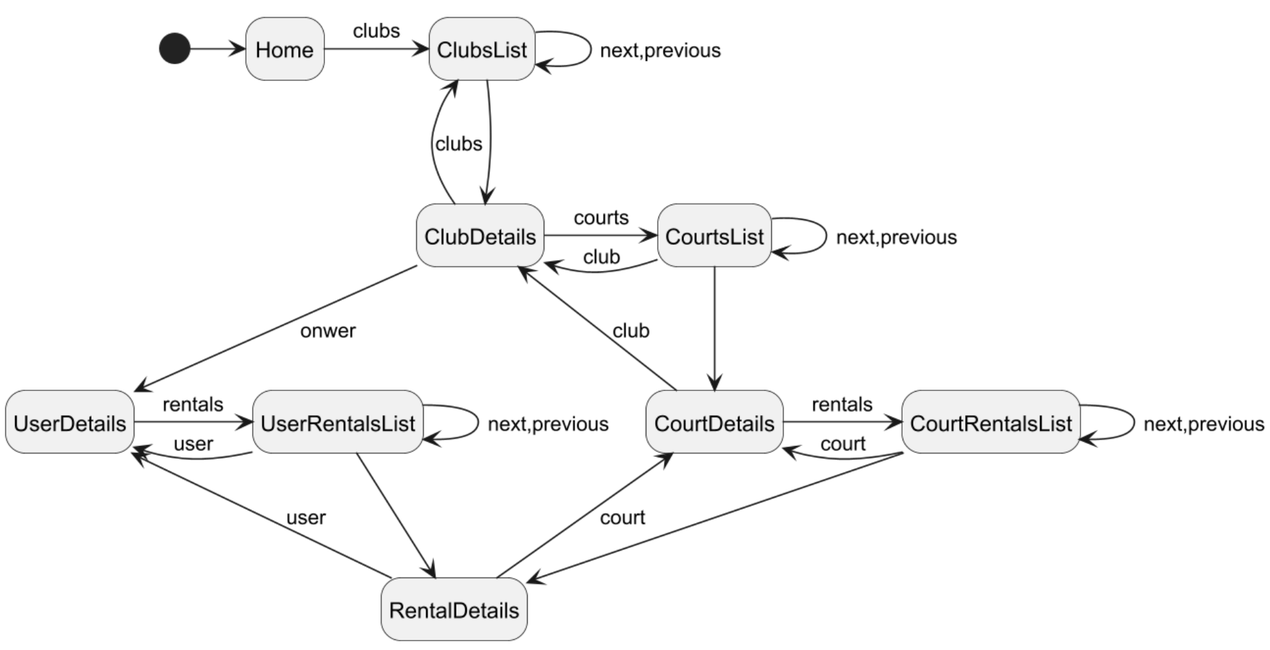
In this repository there are two examples of simple Single Page Applications:

1. A simple one
2. One more complete using a router

To see these two examples working, you must launch the HTTPServer application on `file pt.isel.ls.http.HttpServer.kt` and go to the following URI on your web browser: `http://localhost:9000/`

Note that the routes in `file pt.isel.ls.http.HttpServer.kt` have been updated.

Create HTML views that contain links to ensure the navigability defined in the following graph, changes to the graph should be discuss with the teacher:



All views must have a link to Home

(Click here for a bigger picture)

Report

The technical report created for phase 1 should be updated and/or extended with the relevant technical information. The sections developed for phase 1 can be improved or changed. There should **not** be a separate report for phase 2. The goal is to have a single report, updated through all the project phases.

There isn't any template for phase 2.

Delivery date

The phase 2 must be delivered until 26 of April, through the creation of the 0.2.0 tag on the group´s GitHub repository.

Pages 20

Find a page or section…

Home
Calendar
Demonstration
E Learning Guidelines
Introduction
JDBC
LEIC41D - Groups
LEIC41D - LEIC42D - LEIC43D…
LEIC41N - Groups
LEIC42D - Groups
LEIC43D - Groups
LEIC44D - Groups
Phase 0
Phase 1
Phase 2

Show 5 more pages…

+ Add a custom sidebar

Clone this wiki locally

https://github.com/isel-leic-ls/2425

+ Add a custom footer

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

Phase 3

fbfreitas edited this page on May 15, 2025 · 10 revisions

Phase 3

Introduction

This document describes the requirements for the third phase of the Software Laboratory project.

Requirements

Additional Operations

- Add an operation to search clubs by name. It should support partial names (e.g., PadelLisb or SuperPa)

Single Page Application

When using the web application, in no situation the (human) user will have to know and/or insert any id.

Views

- Update the Home view to have an option to search clubs by name.
- Create a new view to show the available hours in a court in a specific date.
- Update Court details view to have an option to search rentals by date.
- Add the necessary views to create a club, a court and a rental. Also add support for delete and update of a given rental.

Note: Example using HTML form

Refactoring code

This phase should be also used to critically review all code baseline, and start refactoring to improve code quality. Also, more tests should be produced ir order to increase the overall coverage.

Report

The technical report created in the previous phases should be updated and/or extended with the relevant technical information. Remember the goal is to have a single report, updated through all the project phases.

Delivery date

The phase 3 must be delivered until the end of week 13 , through the creation of the 0.3.0 tag on the group´s GitHub repository.

Pages 20
Find a page or section…
Home
Calendar
Demonstration
E Learning Guidelines
Introduction
JDBC
LEIC41D - Groups
LEIC41D - LEIC42D - LEIC43D…
LEIC41N - Groups
LEIC42D - Groups
LEIC43D - Groups
LEIC44D - Groups
Phase 0
Phase 1
Phase 2
Show 5 more pages…
+ Add a custom sidebar
Clone this wiki locally
https://github.com/isel-leic-ls/2425
+ Add a custom footer

© 2026 GitHub, Inc.

Phase 4

fbfreitas edited this page on May 29, 2025 · 6 revisions

Phase 4

Introduction

This document describes the requirements for the fourth and final phase of the Software Laboratory project.

Requirements

Additional Operations

- Update the create user operation, to receive a password.
- Create a new operation that receives the user name and password, and returns the user token and id

Single Page Application

- Produce views to create a new user (Sign Up) and to obtain the user's token (Login).
- The received token should be stored until the user closes the browser window/tab or explicitly executes logout.

Hosting on Render

Host the application on the Render.

1. Create a Render free account. Render is an example of a Platform-as-a-Service (PaaS) provider.
2. On the Render web site, create a new PostgreSQL instance
   - Give it the `Name`: `db-ls-2425-2-<turma>-g<número-do-grupo>`;
   - You can leave the `Database` and `User` fields empty;
   - Select the "Frankfurt(EU Central)" region;
   - Select the Free instance type, and click `Create Database`;
   - Wait for the creation of the database;
   - Create the tables in the render Postgres database, use psql, pgAdmin or a similar tool.
     - **Note that the connection URL must be in the JDBC format (ex. `jdbc:postgresql://<hostname>.frankfurt-postgres.render.com:<port>/<database>?user=<username>&password=<password>`), so you will need to build it from the information provided in the render dashboard.**
3. On the command line, run `gradlew build` in the project root directory.
4. Create a docker free account; for more information about the Docker environment, check here.
5. Install Docker Desktop
6. Add the `Dockerfile` to the root of the group repository, this file should contain all the information to create the docker image, see our example here.
7. On the command line, do `docker login`.
8. Create the docker image , on the command line, do `docker build -t <docker-username>/img-ls-2425-2-<turma>-g<número-do-grupo> .` (for Mac computers with M1 or M2 include --platform linux/amd64)
9. Push the image to docker hub, on the command line, do `docker push <docker-username>/img-ls-2425-2-<turma>-g<número-do-grupo>`
10. On the Render web site, create a new Web Service with the name `service-ls-2425-2-<turma>-g<número-do-grupo>`
    - Select `Deploy an existing image from a registry`
    - Insert Image URL, `docker.io/<docker-username>/img-ls-2425-2-<turma>-g<número-do-grupo>`
    - Name your application and select the "Frankfurt(EU Central)" region
    - Select the free instance type
    - Add a `JDBC_DATABASE_URL` environment variable to your the new Web Service, using the credentials available in render Postgres dashboard (ex. `jdbc:postgresql://<hostname>.frankfurt-postgres.render.com:<port>/<database>?user=<username>&password=<password>`)

- Click create service and wait for the service creation

Note: The service port must be obtained from the environment variable `PORT`.

To launch application in developer machine:

- `docker run -d -p 9000:8080 --env PORT=8080 --env JDBC_DATABASE_URL="jdbc:postgresql://host.docker.internal/<database>?user=<username>&password=<password>" <docker-username>/img-ls-2425-2-<turma>-g<número-do-grupo>`

Refactoring code

This phase should be used to critically review all code baseline, and refactoring to improve code quality should be a goal. Also, more tests should be produced ir order to increase the overall coverage.

Report

The technical report should be updated and/or extended with the relevant technical information. The sections developed in the previous phases can be improved or changed. **There should not be a separate report for each phase.**

Delivery

The completed project must be delivered until 15 June, via the creation of a `1.0.0` tag on the GitHub repository.

Pages 20

Find a page or section…

Home
Calendar
Demonstration
E Learning Guidelines
Introduction
JDBC
LEIC41D - Groups
LEIC41D - LEIC42D - LEIC43D…
LEIC41N - Groups
LEIC42D - Groups
LEIC43D - Groups
LEIC44D - Groups
Phase 0
Phase 1
Phase 2
Show 5 more pages…

+ Add a custom sidebar

Clone this wiki locally

https://github.com/isel-leic-ls/2425

+ Add a custom footer

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information